

Materialized View

A Materialized View is a snapshot of data.

Changes in the base table are not visible until REFRESH MATERIALIZED VIEW is executed.

Insert,update,delete operation cannot be done..

```
CREATE MATERIALIZED VIEW sample_schema.mv_users AS  
SELECT * FROM sample_schema.users;
```

```
SELECT count(*) FROM sample_schema.mv_users  
SELECT count(*) FROM sample_schema.users  
insert into sample_schema.users values(9,'user_name','departmen')
```

```
REFRESH MATERIALIZED VIEW sample_schema.mv_users;
```

PostgreSQL does not support Indexed Views.

PREPARE statement:

While using function or stored procedure with parameters (p_id, p_name, etc.), normally do not need PREPARE. The parameters are passed directly to the function/procedure.

PREPARE creates a prepared statement that PostgreSQL can execute multiple times with different

parameter values.

PREPARE is mainly used for repeatedly executing the same SQL statement within a session.

```
PREPARE get_user(INT) AS  
SELECT *  
FROM sample_schema.users  
WHERE user_id = $1;
```

```
EXECUTE get_user(1);
```

Preventing for sqlinjection

Access control:

-- Create User

```
CREATE USER analyst WITH PASSWORD 'password123';
```

-- Create Role

```
CREATE ROLE reporting_role;
```

-- Grant Role to User

```
GRANT reporting_role TO analyst;
```

-- Grant Database Access

```
GRANT CONNECT ON DATABASE perficient_training TO analyst;
```

```
-- Grant Schema Access  
GRANT USAGE ON SCHEMA sample_schema TO analyst;
```

```
-- Grant SELECT Permission  
GRANT SELECT ON users TO analyst;
```

```
REVOKE SELECT  
ON users  
FROM analyst;
```

```
-- Check User Privileges  
SELECT grantee,  
       table_name,  
       privilege_type  
FROM information_schema.role_table_grants  
WHERE grantee = 'analyst';
```

```
GRANT SELECT(name, email)  
ON users  
TO analyst;
```

```
GRANT EXECUTE  
ON FUNCTION get_user(INT)
```

TO analyst;

GRANT EXECUTE
ON PROCEDURE greet_user()
TO analyst;

Enryption and Encoding and Data Masking

CREATE EXTENSION IF NOT EXISTS pgcrypto;

```
update sample_schema.users set credit_card= pgp_sym_encrypt(
    '123456789012',
    'secret_key'
) where user_id=1
```

```
select user_id,credit_card,pgp_sym_decrypt(
    credit_card,
    'secret_key'
) from sample_schema.users where user_id=1
```

enryption and Encoding and
SELECT
LEFT(user_name, 2) ||

```
*****! ||  
RIGHT(user_name, 2) AS name  
FROM sample_schema.users;
```

Creating view for masking:

```
CREATE OR REPLACE VIEW sample_schema.vw_masked_users1 AS  
SELECT  
    user_id,  
    LEFT(user_name, 2) ||  
    *****! ||  
    RIGHT(user_name, 2) AS masked_name  
FROM sample_schema.users;  
  
SELECT * from sample_schema.vw_masked_users1
```